AML LAB ASSIGNMENT - 2

a. Computing distances and similarities, implementation and analysis of K means clustering and Hierarchical clustering, dendrogram construction and linkage methods.

Python code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.metrics.pairwise import euclidean_distances, cosine_similarity
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

# 1. Load data and select numerical features
data = pd.read_csv("income_data.csv")
X = data[["income", "happiness"]]

# 2. Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Calculate distances and similarities for first five rows
print("Euclidean distance:\n",
      np.round(euclidean_distances(X_scaled[:5]), 3))
print("Cosine similarity:\n",
      np.round(cosine_similarity(X_scaled[:5]), 3))

# 4. K-Means clustering
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)
data["KMeans_Cluster"] = kmeans_labels

print("\nK-Means silhouette score:",
      silhouette_score(X_scaled, kmeans_labels))

plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1],
            c=kmeans_labels, cmap="viridis")
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            color="red", marker="X", s=150, label="Centroids")
plt.xlabel("Standardized Income")
plt.ylabel("Standardized Happiness")
plt.title("K-Means Clustering")
plt.legend()
plt.show()

# 5. Hierarchical clustering with different linkage methods
methods = ["single", "complete", "average", "ward"]

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for method, ax in zip(methods, axes.ravel()):
    Z = linkage(X_scaled, method=method)
    dendrogram(Z, truncate_mode="lastp", p=20, ax=ax)
    ax.set_title(method.capitalize() + " Linkage")
    ax.set_xlabel("Clusters / Samples")
    ax.set_ylabel("Distance")
```

```
plt.tight_layout()
plt.show()

# 6. Hierarchical clustering using Ward linkage
Z = linkage(X_scaled, method="ward")
hier_labels = fcluster(Z, t=3, criterion="maxclust")
data["Hierarchical_Cluster"] = hier_labels

print("Hierarchical silhouette score:",
      silhouette_score(X_scaled, hier_labels))

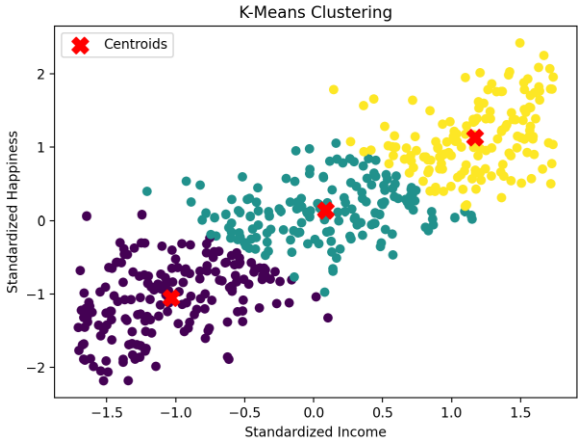
plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1],
            c=hier_labels, cmap="plasma")
plt.xlabel("Standardized Income")
plt.ylabel("Standardized Happiness")
plt.title("Hierarchical Clustering - Ward Linkage")
plt.show()
```

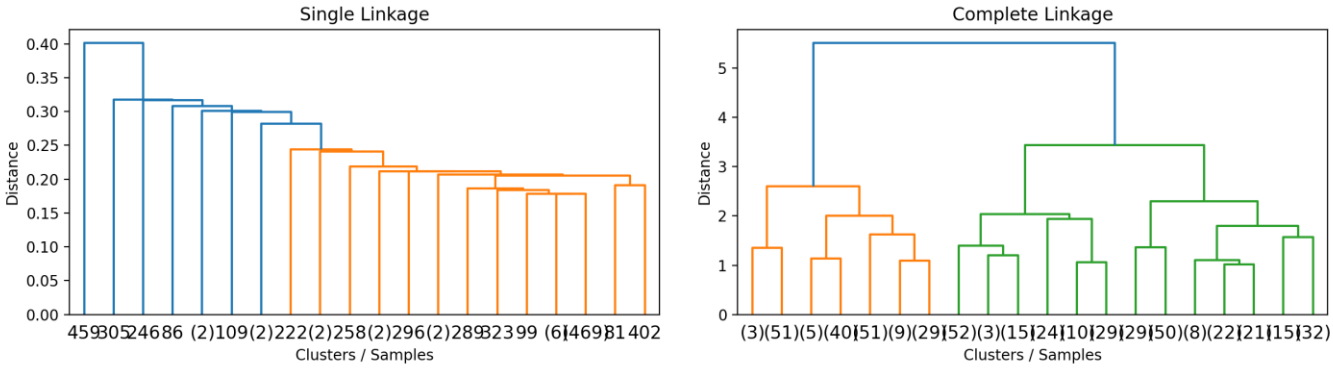
The income and happiness values are standardized, then Euclidean distance and cosine similarity are calculated. K-Means divides the observations into three groups. Hierarchical clustering is tested with single, complete, average and Ward linkage; dendrograms show how clusters merge. Silhouette scores are used to assess cluster separation.

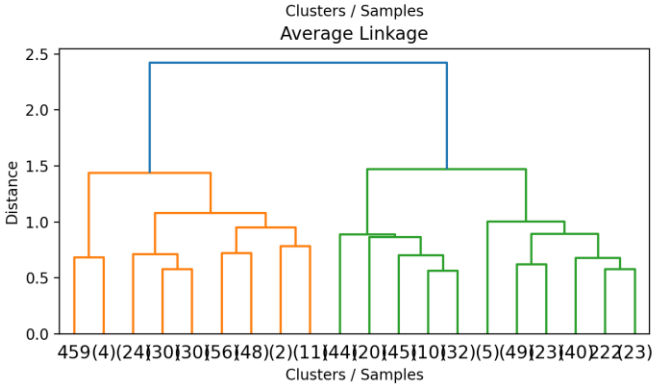
- K-Means is a partition-based clustering algorithm that uses cluster centroids.
- A dendrogram displays the order and distance at which hierarchical clusters merge.
- Ward linkage merges clusters to minimize the increase in within-cluster variance.
- A higher silhouette score generally indicates better-separated clusters.

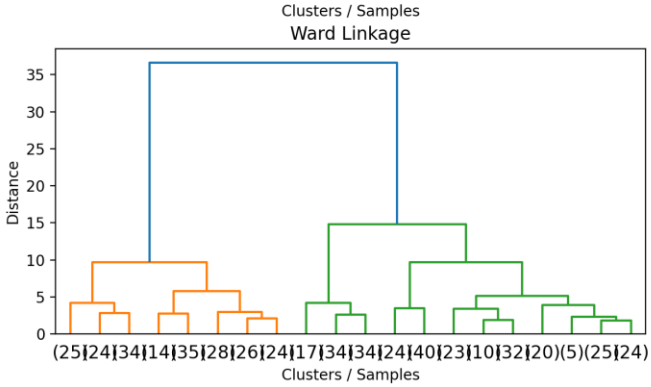
Output:

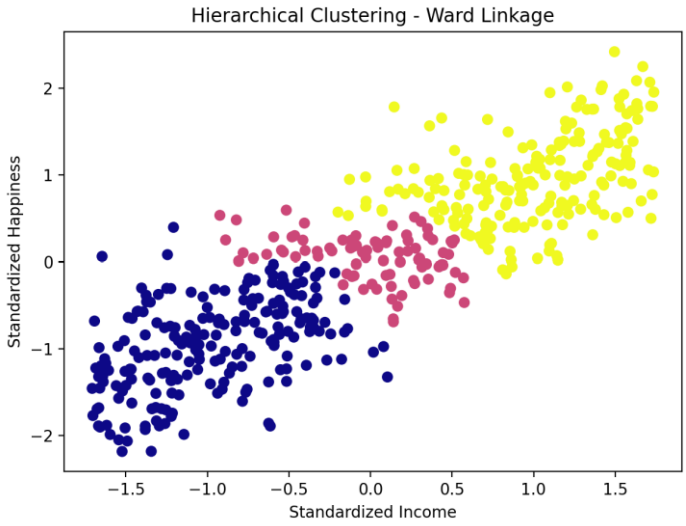
```
Euclidean distance:
 [[0.    1.012 1.709 0.5   2.991]
 [1.012 0.    0.815 1.111 1.979]
 [1.709 0.815 0.    1.602 1.483]
 [0.5   1.111 1.602 0.    3.017]
 [2.991 1.979 1.483 3.017 0.   ]]
Cosine similarity:
 [[ 1.    -0.504 -0.992  0.819 -0.935]
 [-0.504  1.     0.388 -0.908  0.778]
 [-0.992  0.388  1.    -0.738  0.881]
 [ 0.819 -0.908 -0.738  1.    -0.97 ]
 [-0.935  0.778  0.881 -0.97   1.   ]]
```











b. Through coding, understand how ensemble methods in machine learning work, implementation and analysis of bagging techniques such as Random Forest and boosting algorithms.

Python code

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import ConfusionMatrixDisplay
```

```
# 1. Load dataset
data = pd.read_csv("Social_Network_Ads(2).csv")

# 2. Prepare features and target
X = data[["Gender", "Age", "EstimatedSalary"]].copy()
X["Gender"] = X["Gender"].map({"Female": 0, "Male": 1})
y = data["Purchased"]

# 3. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Define ensemble models
models = {
    "Random Forest (Bagging)": RandomForestClassifier(
        n_estimators=100, random_state=42
    ),
    "AdaBoost": AdaBoostClassifier(
        n_estimators=100, random_state=42
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100, learning_rate=0.1, random_state=42
    )
}

# 5. Train and evaluate each model
results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    results[name] = acc

    print("\n", name)
    print("Accuracy:", acc)
    print(classification_report(
        y_test, y_pred,
        target_names=["Not Purchased", "Purchased"]
    ))

    ConfusionMatrixDisplay.from_predictions(
        y_test, y_pred,
        display_labels=["Not Purchased", "Purchased"],
        cmap="Blues"
    )
    plt.title(name)
    plt.show()

# 6. Compare model accuracy
plt.figure(figsize=(8, 5))
plt.bar(results.keys(), results.values())
plt.ylabel("Accuracy")
plt.title("Comparison of Ensemble Models")
plt.xticks(rotation=15)
plt.ylim(0, 1)
plt.tight_layout()
plt.show()

print("\nAccuracy comparison:")
for name, score in results.items():
    print(name, ":", score)
```

This experiment predicts customer purchases using ensemble classifiers. Random Forest uses bagging: multiple decision trees are trained on bootstrap samples and their outputs are combined. AdaBoost and Gradient Boosting build models sequentially, with later learners trying to improve on earlier errors. Their test accuracy and classification reports are compared.

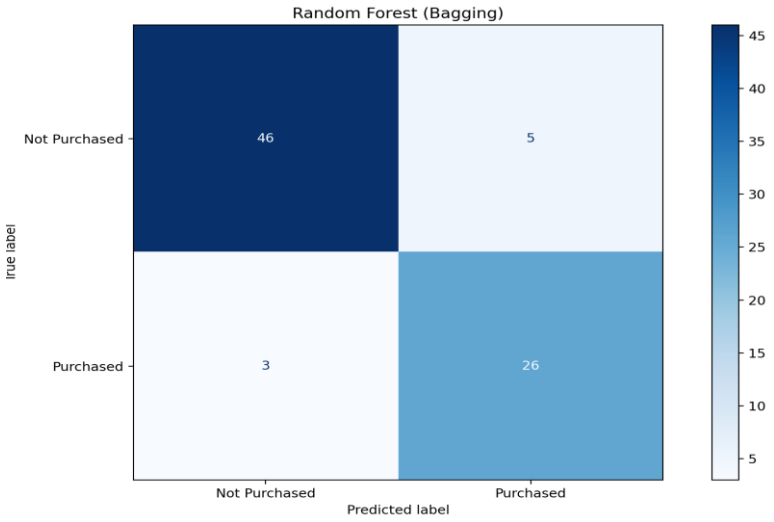
- Bagging trains multiple learners independently on resampled training data.
- Random Forest also uses random subsets of features when splitting tree nodes.
- Boosting builds learners sequentially to improve the overall prediction.
- Accuracy alone may not tell the full story; also inspect precision, recall and F1-score.

Output:

```
 Random Forest (Bagging)
Accuracy: 0.9
              precision    recall  f1-score   support

Not Purchased       0.94      0.90      0.92        51
    Purchased       0.84      0.90      0.87        29

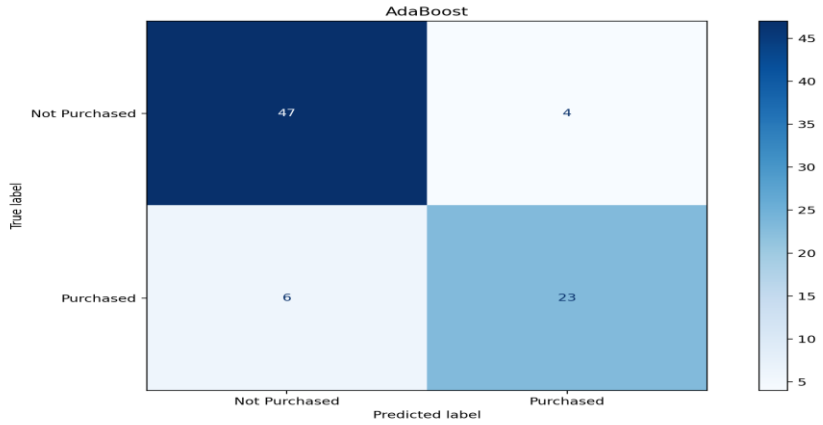
     accuracy                           0.90        80
    macro avg       0.89      0.90      0.89        80
 weighted avg       0.90      0.90      0.90        80
```




```
 AdaBoost
Accuracy: 0.875
              precision    recall  f1-score   support

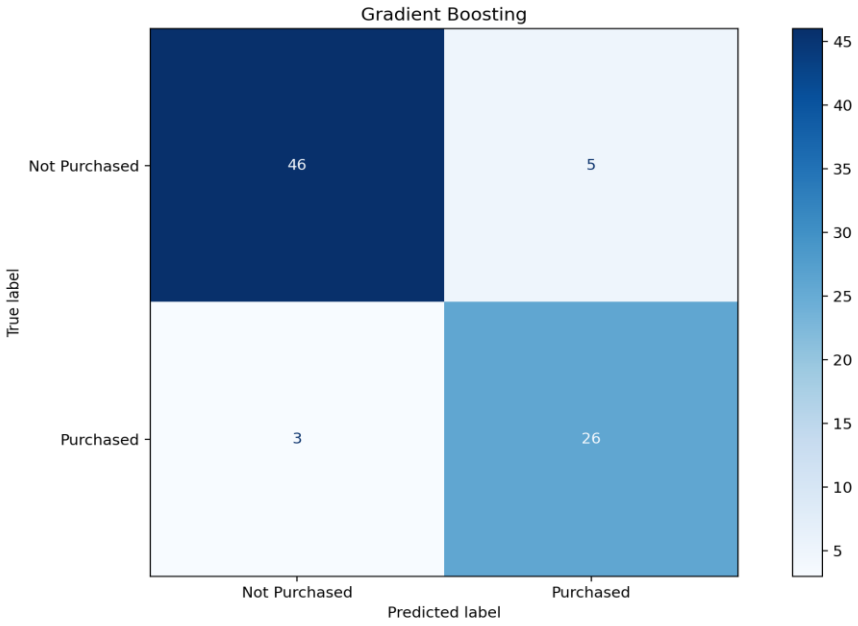
Not Purchased       0.89      0.92      0.90        51
    Purchased       0.85      0.79      0.82        29

     accuracy                           0.88        80
    macro avg       0.87      0.86      0.86        80
 weighted avg       0.87      0.88      0.87        80
```



Gradient Boosting
Accuracy: 0.9

	precision	recall	f1-score	support
Not Purchased	0.94	0.90	0.92	51
Purchased	0.84	0.90	0.87	29
accuracy			0.90	80
macro avg	0.89	0.90	0.89	80
weighted avg	0.90	0.90	0.90	80



c. Implementation of Gaussian Mixture Models (GMM) for clustering tasks.

Python code

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score

# 1. Load dataset
data = pd.read_csv("income_data.csv")
X = data[["income", "happiness"]]

# 2. Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Fit Gaussian Mixture Model
gmm = GaussianMixture(
    n_components=3,
    covariance_type="full",
    random_state=42
)

gmm.fit(X_scaled)

# 4. Assign cluster labels
labels = gmm.predict(X_scaled)
data["GMM_Cluster"] = labels

# 5. Display results
print("First five cluster assignments:")
print(data[["income", "happiness", "GMM_Cluster"]].head())

print("\nConverged:", gmm.converged_)
print("Number of iterations:", gmm.n_iter_)
print("AIC:", gmm.aic(X_scaled))
print("BIC:", gmm.bic(X_scaled))
print("Silhouette Score:", silhouette_score(X_scaled, labels))

# 6. Plot clusters
plt.figure(figsize=(7, 5))
plt.scatter(
    X_scaled[:, 0], X_scaled[:, 1],
    c=labels, cmap="viridis"
)
plt.xlabel("Standardized Income")
plt.ylabel("Standardized Happiness")
plt.title("Gaussian Mixture Model Clustering")
plt.show()

# 7. Show membership probabilities for first five rows
probabilities = gmm.predict_proba(X_scaled)
print("\nCluster membership probabilities (first five):")
print(probabilities[:5])
```

Gaussian Mixture Model represents the data as a mixture of Gaussian distributions. The model estimates the distribution parameters and assigns each observation to the most likely component. Unlike hard assignment methods, GMM can also provide membership probabilities. AIC, BIC and silhouette score help examine model fit and cluster separation.

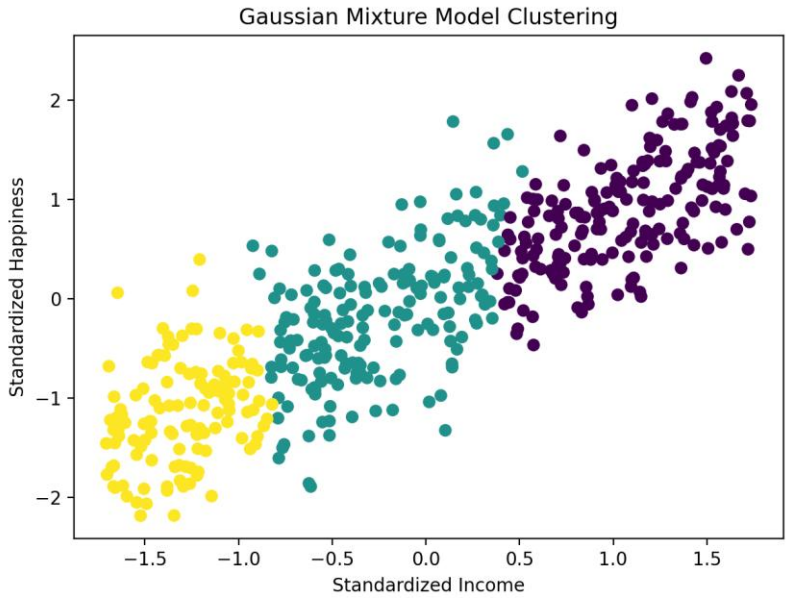
- GMM is a probabilistic, model-based clustering algorithm.

- The number of Gaussian components is specified using n_components.
- predict_proba() returns the probability that each sample belongs to each component.
- AIC and BIC can be used to compare candidate GMM models; lower values are preferred when comparing models on the same data.

Output:

```
First five cluster assignments:
     income  happiness  GMM_Cluster
0  3.862647   2.314489            1
1  4.979381   3.433490            1
2  4.923957   4.599373            1
3  3.214372   2.791114            1
4  7.196409   5.596398            0

Converged: True
Number of iterations: 15
AIC: 2051.7887149845756
BIC: 2123.3689162939945
Silhouette Score: 0.4013148002077116
```




```
Cluster membership probabilities (first five):
[[6.32043355e-03 9.71621964e-01 2.20576024e-02]
 [3.73919114e-01 6.26074513e-01 6.37256063e-06]
 [2.02276849e-01 7.97718957e-01 4.19423673e-06]
 [1.93248461e-04 7.16134016e-01 2.83672736e-01]
 [9.98984371e-01 1.01562941e-03 5.87565070e-19]]
```

d. Implementation and visualization using Principal Component Analysis (PCA).

Python code

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Load dataset
data = pd.read_csv("diabetes(2).csv")

# 2. Separate features and target
X = data.drop("Outcome", axis=1)
y = data["Outcome"]

print("Original data shape:", X.shape)

# 3. Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Apply PCA and reduce to two components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 5. Display transformed data and explained variance
pca_data = pd.DataFrame(
    X_pca,
    columns=["Principal Component 1", "Principal Component 2"]
)

print("\nFirst five PCA rows:")
print(pca_data.head())

print("\nExplained variance ratio:")
print(pca.explained_variance_ratio_)

print("Total variance retained:",
      pca.explained_variance_ratio_.sum())

# 6. Visualize PCA components
plt.figure(figsize=(8, 6))

scatter = plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y,
    cmap="viridis",
    alpha=0.7
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization of Diabetes Dataset")
plt.legend(
    *scatter.legend_elements(),
    title="Outcome"
)
plt.grid()
plt.show()
# 7. Plot explained variance
plt.figure(figsize=(6, 4))
```

```
plt.bar(["PC1", "PC2"],pca.explained_variance_ratio_)
plt.ylabel("Explained Variance Ratio")
plt.title("PCA Explained Variance")
plt.show()
```

PCA is a dimensionality-reduction technique. The eight input features are standardized and transformed into two principal components. These components capture directions of maximum variance in the data and allow the dataset to be visualized in two dimensions. Outcome is used only to colour the points, not to calculate the components.

- PCA is an unsupervised dimensionality-reduction technique.
- Principal components are new, uncorrelated directions formed from combinations of original features.
- Standardization is important when features have different scales.
- Explained variance ratio indicates how much data variance each component captures.

Output:

```
First five PCA rows:
   Principal Component 1  Principal Component 2
0               1.068503               1.234895
1              -1.121683              -0.733852
2              -0.396477               1.595876
3              -1.115781              -1.271241
4               2.359334              -2.184819

Explained variance ratio:
[0.26179749 0.21640127]
Total variance retained: 0.4781987607385657
```

